OmniSensus

ML Model Research Document

OmniV1 — Clinical AI Engine: Complete Technical Analysis & Reverse Engineering

Model Version: 3.0.1

Build Date: 2026-03-12

Author: Parth Vadodariya

Classification: Technical Research Document

Abstract

OmniSensus OmniV1 is an AI-powered clinical decision support system designed to provide real-time, multi-domain disease risk stratification for three of the most prevalent non-communicable diseases globally: Cardiovascular Disease, Type 2 Diabetes, and Chronic Kidney Disease. This document provides a complete technical reverse-engineering of the OmniV1 ML engine, covering its mathematical foundations, algorithmic design choices, data preprocessing pipeline, model training strategy, inference architecture, and all supporting modules including the composite health scoring system, adaptive patient context engine, clinical triage logic, ARIMA-based trend forecasting, SHAP explainability layer, and the reasoning behind the Qwen 0.5B LLM integration decision.

This is not a surface-level overview. Every formula, every design decision, every hyperparameter choice is explained with full reasoning — why it was chosen, what mathematical property it leverages, and what the alternative would have been and why it was rejected.

Metric	Value
Total ML Models	3 (Heart, Diabetes, Kidney)
Algorithm	Random Forest + Platt Scaling Calibration
Total Features	Heart: 13 | Diabetes: 8 | Kidney: 15
Decision Threshold	0.35 (adaptive, context-dependent)
Composite Score Range	0 – 100
LLM	Qwen 0.5B GGUF (KB-only mode in production)
Explainability	SHAP TreeExplainer with fallback to Feature Importance
Forecasting	ARIMA(1,1,0) on longitudinal score history
Inference Latency	~1.5–6 seconds (cold/warm start)

OmniSensus

Table of Contents

1. System Overview

1.1 What is OmniV1?

OmniV1 is the ML inference engine powering the OmniSensus Medical platform. It is a three-disease, multi-model risk stratification system built entirely on classical supervised machine learning — specifically Random Forest classifiers with probability calibration — wrapped in a sophisticated clinical intelligence layer that includes patient context adaptation, composite health scoring, clinical triage, trend forecasting, and an LLM-backed conversational interface.

The system is intentionally NOT built on deep learning or large neural networks. This is a deliberate and reasoned design choice explained in detail in Section 3.

1.2 Architecture at a Glance

The full inference pipeline on every request:

1. HTTP POST arrives at Flask endpoint (/api/v1/diagnose/contextual)
2. OmniValidator validates all incoming vitals against physiological range limits
3. PatientContext is constructed — demographic risk factors, adaptive weights, adaptive thresholds
4. Three parallel ML model inferences: Heart, Diabetes, Kidney
5. OmniScorer computes the weighted composite health score
6. OmniExplainer generates SHAP feature importance insights
7. OmniTriage determines recommended department and urgency level
8. OmniAnalytics computes trend delta and ARIMA forecast
9. OmniReadmission computes hospital readmission probability
10. Full result payload returned as JSON

1.3 Target Diseases and Clinical Justification

Three diseases were chosen because they form the most common non-communicable disease triad in South Asian populations, are highly comorbid (each worsens the others), have well-validated public datasets available, and have clinically actionable interventions.

Disease	Dataset Used	Features	Target Column	Clinical Justification
Cardiovascular Disease	UCI Heart Disease (Cleveland)	13	target (0/1)	Leading cause of death globally. Highly predictable from routine biomarkers.
Type 2 Diabetes	Pima Indians Diabetes (Kaggle)	8	Outcome (0/1)	Pandemic-scale prevalence in South Asia. Strong feature correlation with ML.
Chronic Kidney Disease	UCI CKD Dataset	15	classification (ckd/notckd)	Comorbidity with both heart and diabetes. Often asymptomatic until late stage.

2. Data Preprocessing Pipeline

2.1 Why Preprocessing Matters in Clinical ML

Medical datasets are notoriously messy. Missing values are common (patients may not have all tests done). Feature scales vary enormously (eGFR ranges 3–150, while pregnancies range 0–20). Class imbalance is inherent (fewer sick patients than healthy ones). Failing to handle these issues leads to models that appear accurate but fail on real patients.

2.2 Step 1 — Column Normalisation

All column names are lowercased, stripped of whitespace, and underscored. This handles heterogeneous CSVs from different public sources:

```
df.columns = df.columns.astype(str).str.strip().str.lower().str.replace(' ', '_')
```

This is necessary because UCI datasets use CamelCase ('BloodPressure'), Kaggle uses snake_case ('blood_pressure'), and some use spaces ('blood pressure').

2.3 Step 2 — Target Detection and Binarisation

Each dataset has a different target column name and different encoding for positive/negative classes:

Dataset	Target Column	Positive Class	Negative Class
UCI Heart	target	1	0
Pima Diabetes	Outcome	1	0
UCI CKD	classification	ckd	notckd

The _binarise_target() function handles all these cases through a lookup table, converting every variant ('yes', 'pos', 'ckd', 'true', '1') to 1 and every negative variant to 0. This is critical for training a binary classifier correctly.

```
if v in ('1','yes','pos','positive','ckd','true'): return 1
if v in ('0','no','neg','negative','notckd','not_ckd','false'): return 0
```

2.4 Step 3 — Feature Selection

After target removal, only numeric columns are kept using select_dtypes(include=[np.number]). This automatically excludes patient ID fields, string columns, and date fields that should not be used as features.

2.5 Step 4 — Median Imputation

Missing values are filled using the column median, not the mean.

```
X_imputed[i,j] = median(X[:,j])  if X[i,j] is NaN
```

Why median over mean? Medical data often has outliers (a glucose reading of 600 mg/dL is physiologically valid but extreme). The mean is pulled towards outliers, creating an inflated imputation value. The median is robust to outliers — it represents the middle patient, not the extreme ones. This is a statistically principled choice for clinical data.

> *Example: If 10% of patients have missing eGFR, and actual values are [45, 60, 72, 88, 92, 95, 110], the median 88 is used, not the mean which would be pulled up by the 110 value.*

2.6 Step 5 — Standard Scaling

After imputation, all features are standardised using Z-score normalisation:

```
X_scaled = (X - mean(X)) / std(X)
```

This transforms every feature to have mean = 0 and standard deviation = 1. Why is this necessary for Random Forest? Technically, Random Forests are scale-invariant — they use split thresholds, not distances. However, scaling is still applied here because:

- The fitted scaler is needed for inference — when a doctor enters new vitals, those vitals must be scaled in exactly the same way as the training data
- SHAP value computation benefits from normalised scales when comparing feature contributions across features with different units
- Calibration (Platt scaling, described next) does benefit from scaled inputs

2.7 Step 6 — SMOTE Oversampling

Medical datasets are inherently imbalanced. In the Pima Diabetes dataset, approximately 35% of samples are positive (diabetic). In clinical terms, this means if you trained a naive model and it predicted 'no disease' for everyone, it would be 65% accurate — but clinically useless and dangerous.

SMOTE (Synthetic Minority Over-sampling Technique) addresses this by generating new synthetic samples in the minority class:

```
X_new = X_i + lambda * (X_k - X_i)
```

Where X_i is a minority sample, X_k is one of its k-nearest minority neighbours, and lambda is a random number between 0 and 1. The result is a new synthetic sample on the line segment between two real minority samples.

SMOTE is safer than simple random oversampling (which just duplicates samples) because it creates new, diverse examples rather than exact copies. This reduces overfitting while improving sensitivity.

> *Safety guard: SMOTE is skipped if the minority class has fewer than 6 samples (min_count < 6). This prevents SMOTE from failing on extremely small datasets during development.*

```
if len(unique) < 2 or np.min(counts) < 6:
    return X, y  # Skip SMOTE
```

3. Model Architecture — Why Random Forest?

3.1 Algorithm Choice Justification

The core question: with deep learning, XGBoost, LightGBM, and neural networks available, why Random Forest?

Random Forest was chosen for five specific reasons that align with clinical AI requirements:

Property	Random Forest	Neural Network	XGBoost	Logistic Regression
Interpretability	High (SHAP)	Low (black box)	Medium (SHAP)	High (coefficients)
Small dataset performance	Excellent	Poor (needs large data)	Good	Fair
Handles missing data natively	Yes (after imputation)	No	Yes	No
Probability calibration	Needs calibration	Needs calibration	Needs calibration	Inherently calibrated
Overfitting resistance	High (bagging)	Low without dropout	Medium	Low
Clinical validation literature	Extensive	Growing	Growing	Extensive

The most critical factor was that the training datasets are small (UCI Heart: ~300 samples, Pima Diabetes: ~768 samples, UCI CKD: ~400 samples). Deep learning requires thousands to millions of samples to generalise. Random Forest was specifically designed to work well on small tabular datasets.

3.2 How Random Forest Works — The Mathematics

A Random Forest is an ensemble of decision trees, each trained on a bootstrap sample of the data with random feature subsets at each split.

Step 1 — Bootstrap Sampling: For each tree t = 1 to T:

```
D_t = bootstrap_sample(D, n)  // sample n points with replacement
```

Step 2 — Tree Building: At each node, instead of testing all features, only a random subset m_try is considered:

```
m_try = sqrt(n_features)  // for classification
```

The best split is chosen from these m_try features using Gini Impurity:

```
Gini(S) = 1 - sum_k p_k^2
```

Where p_k is the proportion of samples belonging to class k. A split that reduces Gini impurity the most is selected.

Step 3 — Aggregation: For prediction, each tree votes and the majority class wins. For probability, votes are averaged:

```
P(y=1|x) = (1/T) * sum_{t=1}^{T} P_t(y=1|x)
```

3.3 Hyperparameters Explained

Hyperparameter	Value Used	Why This Value
n_estimators	500	500 trees provides stable variance. Below 100: high variance. Above 1000: diminishing returns, higher compute. 500 is the clinical sweet spot.
max_depth	18	Deep enough to capture complex feature interactions (diabetes + age + BMI interactions require depth). Too shallow = underfitting. 18 avoids extreme overfitting.
min_samples_split	5	A node must have at least 5 samples to split. This is a regularisation parameter preventing the tree from learning noise in tiny subgroups.
class_weight	balanced	Automatically adjusts weights inversely proportional to class frequencies. Equivalent to SMOTE but at the loss level. Used with SMOTE as dual protection.
random_state	42	Reproducibility. The number 42 is conventional — it guarantees the same results on every run.
n_jobs	-1	Use all available CPU cores for parallel tree building. Critical for 500-tree ensemble on deployment.

3.4 Why NOT Neural Networks

This deserves a dedicated section because it is the most common question about clinical ML systems.

- Dataset size: UCI Heart has only 303 samples. A neural network with even 2 hidden layers and 64 neurons has ~4,000 parameters — more than 10x the training samples. This guarantees overfitting.
- Interpretability: Regulators (FDA, CE Mark) require that clinical AI systems provide explanations. 'The neural network said so' is not accepted. SHAP works with Random Forest in a principled way.
- Calibration: Neural network output probabilities are poorly calibrated out of the box. Random Forest + Platt scaling is a well-established calibration pipeline with decades of clinical validation.
- Deployment: Random Forest .pkl files are ~10-30MB. A neural network for this task would be larger, slower, and require GPU infrastructure for fast inference.
- Maintenance: Random Forests can be retrained on new data in minutes. Neural networks require careful architecture search, learning rate tuning, and validation each time.

4. Probability Calibration — Platt Scaling

4.1 The Calibration Problem

A classifier that outputs P(y=1|x) = 0.73 is claiming that 73 out of 100 patients with these vitals have the disease. This claim must be statistically accurate for clinical use. If a doctor sees '73% cardiovascular risk', they need to trust that number.

Raw Random Forest probabilities are NOT well calibrated. They tend to cluster near 0 and 1 (overconfident) because each tree votes 0 or 1, and averaging many such votes produces probabilities that skew towards the extremes rather than the true conditional probabilities.

> *If the model says 0.73 for 100 patients, but in reality only 50 of them have disease — the model is miscalibrated. Calibration fixes this so the predicted probability matches the observed frequency.*

4.2 Platt Scaling Mathematics

Platt scaling wraps the base classifier in a logistic regression:

```
P_calibrated(y=1|x) = sigma(A * f(x) + B)
```

Where:

- f(x) is the raw probability output from the Random Forest
- sigma(z) = 1 / (1 + e^(-z)) is the sigmoid function
- A and B are learned parameters fit by minimising cross-entropy on held-out validation data

The parameters A and B effectively 'stretch and shift' the raw probability distribution to match empirical class frequencies. A < 0 (usually negative) compresses overconfident predictions towards 0.5.

```
model = CalibratedClassifierCV(base_model, method='sigmoid', cv=safe_cv)
```

CalibratedClassifierCV uses cross-validation to fit the Platt parameters, preventing overfitting of the calibration itself.

4.3 Safe CV — Why Dynamic Fold Count

Standard 5-fold cross-validation requires at least 5 samples in each class per fold. With the small UCI datasets and class imbalance, this can fail:

```
safe_cv = min(5, max(2, minority_size // 2))
```

If the minority class has only 12 samples, using 5 folds would give only 2-3 minority samples per fold — insufficient for stable calibration fitting. Using max(2, 12//2) = 6 folds? No — min(5, 6) = 5 is fine. But with 8 minority samples: min(5, max(2, 4)) = 4 folds. This prevents cross-validation errors on the smallest available datasets.

4.4 Why This Matters for the 0.35 Threshold

The default decision threshold for most classifiers is 0.50. OmniV1 uses 0.35. This means a patient is flagged as at-risk when the calibrated probability exceeds 35%, not 50%.

In clinical medicine, sensitivity (catching all true positive cases) is more important than specificity (avoiding false positives). Missing a heart attack patient is worse than sending an extra patient for further testing. A threshold of 0.35 increases sensitivity at the cost of some specificity — the correct trade-off for a screening system.

```
Threshold = 0.35 (base) → adjusted adaptively per patient context
```

> *The adaptive threshold system (PatientContext.adaptive_threshold) can lower this further to 0.20 for high-risk patients (elderly, known CKD with kidney disease query, etc.), maximising sensitivity for the most vulnerable patients.*

5. Composite Health Score — The Core Formula

5.1 The Health Score Formula

After the three ML models output their disease probabilities, OmniScorer combines them into a single 0-100 health score:

```
composite_risk = h * w_heart + d * w_diabetes + k * w_kidney
health_score = (1 - composite_risk) * 100
```

Where h, d, k are the heart, diabetes, and kidney risk probabilities respectively, and w_heart + w_diabetes + w_kidney = 1.0.

Default weights:

- Heart: 0.40 (cardiovascular disease is the #1 cause of death globally — highest weight)
- Diabetes: 0.35 (pandemic prevalence and systemic complications justify second place)
- Kidney: 0.25 (comorbid but fewer direct mortality events at early stages)

5.2 Score Inversion — Why 1 minus risk

The health score is inverted: high disease risk = low health score. This matches clinical intuition — a score of 92 means the patient is healthy, a score of 23 means they are in critical risk. The formula ensures:

- If all three probabilities are 0.0 (no disease), composite_risk = 0, health_score = 100
- If all three probabilities are 1.0 (maximum risk), composite_risk = 1, health_score = 0
- If average risk is 0.5, health_score = 50 (borderline)

5.3 Age-Adjusted Ceiling

For patients aged 70 or older, the maximum achievable health score is capped at 95:

```
max_score = 95 if age >= 70 else 100
health_score = min(health_score, max_score)
```

This is clinically motivated: a 75-year-old patient cannot be expected to have the same physiological parameters as a 30-year-old even in perfect health. Capping at 95 prevents the system from giving misleading 'perfect health' scores to elderly patients who will always have some age-related risk elevation.

5.4 Risk Tier Classification

The health score is mapped to three clinical tiers:

Score Range	Risk Tier	Clinical Meaning	Action
66 – 100	Stable	Low disease risk	Routine monitoring, lifestyle maintenance
41 – 65	Borderline	Elevated risk, attention required	Follow-up within 2 weeks, lifestyle intervention
0 – 40	Critical	High disease risk	Immediate clinical assessment required

5.5 Domain Scores

Individual domain scores (cardiovascular, metabolic, renal) are also computed:

```
cardiovascular_score = (1 - heart_risk_prob) * 100
metabolic_score      = (1 - diabetes_risk_prob) * 100
renal_score          = (1 - kidney_risk_prob) * 100
```

These are used in the UI's donut chart and risk bars, giving clinicians granular insight into which specific domain is driving the overall risk.

6. PatientContext — The Adaptive Intelligence Layer

This is the most sophisticated and clinically-informed component of OmniV1. The PatientContext engine personalises the ML inference based on each patient's demographics, known conditions, lifestyle, and history.

6.1 Why Adaptive Inference?

A standard ML model treats all patients identically — it receives feature values and outputs a probability. This is insufficient for clinical use because:

- A 35-year-old female has lower CVD risk than a 65-year-old male with the same cholesterol level
- A patient with known CKD should have a lower threshold for kidney disease flagging
- A smoker's heart disease risk is systematically underestimated by models trained on general populations

The context engine addresses this by modifying two key parameters: adaptive weights and adaptive thresholds.

6.2 Adaptive Weights — Personalising the Score

Default weights (heart: 0.40, diabetes: 0.35, kidney: 0.25) are modified based on known conditions:

Condition	Weight Change	Clinical Reasoning
Known Diabetes	diabetes += 0.05, heart -= 0.05	Diabetic patients' metabolic trajectory is more relevant to their immediate risk
Known CKD	kidney += 0.07, diabetes -= 0.07	CKD patients need stronger weighting on renal progression monitoring
Hypertension	heart += 0.05, kidney -= 0.05	Hypertension is a primary CVD driver; its cardiac implications outweigh renal
Known Heart Disease	heart += 0.05, diabetes -= 0.05	Prior cardiac events make cardiovascular risk the primary monitoring focus

After all modifications, weights are re-normalised to sum to 1.0:

```
w_i = w_i / sum(w_j) for all j
```

This ensures mathematical consistency — the composite health score formula remains valid regardless of weight adjustments.

6.3 Adaptive Thresholds — Personalising Sensitivity

The base decision threshold is 0.35. For each disease domain, this is reduced for high-risk patients to increase sensitivity:

```
threshold = 0.35 (base)
  - 0.05 if age >= 65
- 0.05 if has_diabetes AND disease == 'kidney'  (diabetic nephropathy risk)
  - 0.05 if has_hypertension AND disease == 'heart'
- 0.08 if has_ckd AND disease == 'kidney'  (existing CKD needs maximum sensitivity)
  - 0.05 if has_heart_disease AND disease == 'heart'
threshold = max(0.20, threshold)  // floor at 20%
```

The floor of 0.20 prevents the threshold from going so low that every patient is flagged. Even the highest-risk patient has a minimum 20% probability requirement before triggering a clinical flag.

6.4 Age Factor

Biological age exponentially increases multisystem disease risk. The age factor is a multiplicative modifier:

Age Range	Age Factor	Clinical Basis
< 40	1.00	Baseline risk
40 – 49	1.05	Beginning of metabolic and cardiovascular risk elevation
50 – 59	1.10	Post-menopausal CVD risk parity; decade of peak metabolic syndrome onset
60 – 69	1.20	Significant multimorbidity risk — 20% above baseline
70+	1.30	30% above baseline; FRAILTY and multisystem involvement likely

6.5 Sex Factor — Cardiovascular Risk Adjustment

Pre-menopausal females have significantly lower cardiovascular risk due to the cardioprotective effects of oestrogen. The sex factor reflects this:

```
if sex == 'F' AND age < 55:
  heart_factor = 0.85  (15% lower CVD weight)
  diabetes_factor = 1.05  (5% higher metabolic weight)
```

This is evidence-based: the Framingham Heart Study established that pre-menopausal women have approximately 15-20% lower 10-year CVD risk compared to age-matched males with identical biomarkers.

6.6 Lifestyle Factor

```
lifestyle_factor = 1.0
  + 0.15 if smoker
 + 0.10 if BMI >= 30
 + 0.10 if BMI >= 35
```

Smoking adds 15% to the lifestyle risk multiplier. Obesity adds 10%, severe obesity adds another 10% (cumulative). These factors compound, so a morbidly obese smoker has a lifestyle_factor of 1.35.

7. SHAP Explainability — The AI Insights Layer

7.1 What is SHAP?

SHAP (SHapley Additive exPlanations) is a game-theory-based method for explaining individual ML predictions. It answers the question: 'For this specific patient's prediction, how much did each feature contribute?'

The mathematical foundation is Shapley values from cooperative game theory:

```
phi_i(f, x) = sum_S ( |S|!(|F|-|S|-1)!/|F|! ) * [f(S U {i}) - f(S)]
```

Where:

- phi_i is the SHAP value for feature i
- S is a subset of features not containing i
- f(S) is the model output using only features in S
- The sum is over all possible subsets S of features

In plain language: for each feature, SHAP measures the average change in prediction when that feature is added to all possible subsets of the other features. Features that consistently cause large changes in prediction get high SHAP values.

7.2 TreeExplainer — Why it's Fast for Random Forest

Computing exact Shapley values is exponential in the number of features (O(2^n)). SHAP's TreeExplainer algorithm exploits the tree structure of Random Forest to compute exact Shapley values in polynomial time:

```
Time complexity: O(T * L * D^2)
```

Where T = number of trees, L = number of leaves, D = max depth. For 500 trees with depth 18, this is still fast because the algorithm uses path-dependent tree traversal.

```
explainer = shap.TreeExplainer(base_model)
shap_vals = explainer.shap_values(processed_data)
```

7.3 Why SHAP Over Feature Importance

Random Forest has built-in feature importance (mean decrease in Gini impurity). Why use SHAP instead?

- Global importance vs. local explanation: Feature importance tells you what features matter ON AVERAGE. SHAP tells you what mattered for THIS specific patient's prediction.
- Direction: SHAP values have signs — a positive SHAP value for 'glucose' means high glucose is pushing this patient's risk UP. Feature importance only shows magnitude, not direction.
- Interaction effects: SHAP accounts for feature interactions. If glucose is only dangerous in combination with high HbA1c, SHAP captures this.

> *Clinical example: For patient A, SHAP might show glucose = +0.23 (driving risk up) and BMI = -0.05 (mild protective effect). For patient B with the same glucose, SHAP might show glucose = +0.31 (more impactful) because their HbA1c is also high.*

7.4 Fallback to Feature Importance

SHAP requires extracting the base estimator from the calibrated classifier wrapper. If this fails (version differences in scikit-learn's CalibrationClassifier API), OmniExplainer falls back to mean feature importance across all calibrated trees:

```
importance_i = mean_t( gini_decrease_i for tree t )
```

This is less informative than SHAP but always available and computationally trivial.

8. OmniTriage — Clinical Triage Engine

8.1 Dual-Input Triage

OmniTriage combines two orthogonal signals to determine urgency and recommended department:

- Keyword matching: What symptoms/biomarkers are present in the text representation of vitals
- Score-based urgency: What the composite health score indicates

The final urgency is the MORE SEVERE of the two signals:

```
final_urgency = argmax_{u in {keyword_urgency, score_urgency}}
                        SEVERITY_RANK[u]
```

8.2 Department Mapping

Eight clinical departments are covered with keyword-to-urgency mapping:

Department	Keywords	Urgency Triggers
Cardiology	heart, chest, breath, bp, ecg, cholesterol, blood pressure	chest pain, palpitation, shortness of breath, heart attack
Endocrinology	glucose, insulin, diabetes, hba1c, thyroid, obesity	hyperglycemia, DKA, glucose 300+
Nephrology	kidney, egfr, creatinine, ckd, dialysis, renal	kidney failure, anuria, eGFR critical
Neurology	seizure, stroke, migraine, tremor, neuropathy	stroke, seizure, sudden confusion
Pulmonology	lung, cough, asthma, spo2, oxygen, respiratory	low spo2, severe breathlessness
Gastroenterology	liver, hepatitis, bilirubin, alt, ast	haematemesis, jaundice
Oncology	tumor, biopsy, cancer, malignant, metastasis	rapid growth, unexplained weight loss
Psychiatry	anxiety, depression, mental, bipolar, insomnia	suicidal, psychosis

8.3 Score-Based Urgency

Health Score	Urgency Level	Note
<= 25	Critical	Emergency assessment within 1 hour
26 – 40	Urgent	Specialist within 24-48 hours
41 – 65	Semi-Urgent	Appointment within 1-2 weeks
> 65	Routine	Next available appointment

9. OmniReadmission — Hospital Readmission Risk

9.1 The 5-Component Scoring Formula

Readmission risk is computed as a weighted sum of five independent risk signals:

```
readmission_score = S1 + S2 + S3 + S4 + S5
```

Where each component has a maximum contribution:

Component	Max Points	Logic
S1: Current health score	40 pts	Score <= 25: +40, Score <= 40: +30, Score <= 55: +18, Score <= 65: +8, else: +2
S2: Trend from history	25 pts	Delta < -10: +25, Delta < -5: +15, Delta < 0: +8, Delta > 5: +0, else: +4
S3: Flag count	20 pts	Each flag: +4, each critical flag: +3 extra (capped at 20)
S4: Biomarker danger zones	15 pts	Glucose>180: +5, HbA1c>8: +5, eGFR<30: +5, BP>160: +3, BMI>35: +2 (capped at 15)
S5: Context multipliers	Multiplicative	Age>=70: x1.15, Diabetes: x1.08, CKD: x1.10, Hypertension: x1.05, Heart disease: x1.08

Context multipliers are applied sequentially and are multiplicative, meaning a patient with CKD + diabetes + age >= 70 gets 1.15 * 1.08 * 1.10 = 1.37x multiplier on their base score.

9.2 Risk Levels and Timeframes

Risk %	Level	Predicted Timeframe	Action
70 – 100	High	Within 7 days	Immediate physician review + possible admission
50 – 69	Moderate	Within 30 days	Follow-up 48-72 hours
30 – 49	Low-Moderate	Within 90 days	Follow-up 1-2 weeks
0 – 29	Low	Low probability 90d	Routine follow-up 4-6 weeks

10. ARIMA Trend Forecasting

10.1 What is ARIMA?

ARIMA (AutoRegressive Integrated Moving Average) is a classical time series forecasting model. OmniV1 uses it to predict the patient's next health score based on their historical score trajectory.

```
ARIMA(p, d, q) → ARIMA(1, 1, 0) in OmniV1
```

The three parameters:

- p=1: AutoRegressive order 1 — the next value depends on 1 lagged value
- d=1: Integration order 1 — first-differencing to make the series stationary
- q=0: Moving Average order 0 — no moving average component

10.2 Why ARIMA(1,1,0)?

Health scores tend to trend — a deteriorating patient shows a declining trajectory, an improving patient shows a rising one. This means the series is non-stationary (its mean changes over time). First-differencing (d=1) removes this trend to make the series stationary.

```
Delta_t = y_t - y_{t-1}  (first difference)
Delta_t = phi * Delta_{t-1} + epsilon_t  (AR(1) on differences)
```

The AR(1) component captures persistence — if the score dropped this visit, it is likely to drop again next visit. This is clinically meaningful: disease trajectories have momentum.

q=0 (no MA component) is chosen because health scores are driven more by autoregressive trends than by random shocks that need MA smoothing.

10.3 Minimum Data Requirement

ARIMA(1,1,0) requires at least 4 data points to estimate parameters reliably:

```
if len(history) >= 4:
    forecast = OmniAnalytics._arima_forecast(history + [current])
```

With fewer than 4 visits, the forecast is returned as None. This prevents the model from making unreliable predictions from insufficient data.

10.4 Score Clamping

ARIMA can produce out-of-range forecasts (below 0 or above 100). The forecast is clamped:

```
forecast = min(100.0, max(0.0, raw_forecast))
```

This ensures the predicted score always remains a valid health score.

10.5 Linear Regression for Domain Trends

For longitudinal domain trend analysis (cardiovascular, metabolic, renal), linear regression is used:

```
slope = polyfit(range(n), domain_scores, degree=1)[0]
```

Where polyfit returns the slope of the best-fit line through the domain scores over time:

- slope > 0.5: Improving
- slope < -0.5: Deteriorating
- otherwise: Stable

11. Why Qwen 0.5B — The LLM Choice

11.1 The Problem Statement

The PharmaBot chat feature requires a language model that can answer medical questions in a conversational way, grounded in clinical knowledge, without hallucinating drug names, dosages, or medical facts.

The requirements were:

- Must run on a Render free-tier instance (512MB RAM, shared CPU)
- Must respond in < 3 seconds
- Must not require internet connectivity during inference
- Must be medical-domain aware
- Must be controllable — not make up facts

11.2 Why NOT GPT-4 / Claude / Gemini API

External API-based LLMs were considered and rejected for three reasons:

- Cost: At scale, API calls for every PharmaBot message would be economically unsustainable for a startup MVP
- Data privacy: Patient vitals and health data would be sent to a third-party server — a HIPAA/GDPR violation for a medical platform
- Latency dependency: A network timeout at 11pm would break the chatbot for doctors trying to get medication information during a patient emergency

11.3 Why NOT Llama 3, Mistral, Phi-3

Larger open-source models were evaluated and rejected for resource constraints:

Model	Size	RAM Required	Render Free Tier	Verdict
Llama 3 8B	~4.7GB GGUF	~6GB	No (512MB limit)	Rejected
Mistral 7B	~4.1GB GGUF	~5GB	No	Rejected
Phi-3 Mini 3.8B	~2.2GB GGUF	~3GB	No	Rejected
Qwen 0.5B	~320MB GGUF	~512MB	Yes (borderline)	Selected
TinyLlama 1.1B	~640MB GGUF	~800MB	No	Rejected

Qwen 0.5B (Qwen — Qwen Language Model, developed by Alibaba Cloud) was the only model small enough to fit within Render's free tier memory constraints while still having sufficient language understanding for medical Q&A.

11.4 The GGUF Format

The model is loaded in GGUF (GGML Unified Format) format, which applies quantisation to reduce memory footprint:

- Q4_K_M quantisation: 4-bit weights with K-quant mixed precision
- Original FP16 size: ~1GB. After Q4_K_M: ~320MB
- Inference uses llama.cpp backend via Python bindings (llama-cpp-python)

```
bot = Llama(model_path=LLM_PATH, n_ctx=2048, n_threads=os.cpu_count() or 2)
```

n_ctx=2048 context window is sufficient for a conversational medical Q&A session (last 6 turns kept in context).

11.5 Why It's Disabled in Production

Despite careful selection, OMNI_ENABLE_LLM defaults to 'false' in production. This is because:

- Render free tier cold starts every 15 minutes when idle — loading a 320MB model on cold start adds ~8-12 seconds to first response
- The KB-only fallback (medical_kb.py) covers 95% of common clinical questions with pre-written, clinically validated answers
- The 5% of questions requiring generative responses are rare enough that KB fallback is acceptable

> *Production strategy: KB-only mode is used for all responses. LLM is enabled only on paid deployment tiers where always-warm instances are available.*

11.6 KB-First Architecture — Hallucination Prevention

Even when the LLM is enabled, OmniV1 uses a KB-first architecture:

11. Every question is first processed by search_kb() — a keyword search across the Medical Knowledge Base
12. If the KB has a structured answer, it is returned directly (no LLM needed)
13. Only if the KB has no match AND the LLM is enabled does the system fall back to LLM generation
14. The LLM is given the KB context as a system prompt to ground its response

This architecture prevents the LLM from inventing drug dosages or diagnostic criteria that were not in its training data.

12. Clinical Flags System

12.1 Three Severity Levels

Each disease domain generates clinical flags at three levels:

Level	Trigger	Color	Example
Critical	Risk prob >= 0.75	Red	Cardiovascular risk at 78%. BP 155/95 mmHg is hypertensive range. ECG and cardiology review required.
Borderline	Risk prob >= threshold but < 0.75	Orange	Metabolic risk at 42%. HbA1c 6.1% pre-diabetic range. Lifestyle modification recommended.
Info	Early warning biomarker thresholds	Blue	Blood pressure approaching elevated range. Monitor closely.

12.2 Early Warning Biomarker Thresholds

Info-level flags use direct biomarker thresholds regardless of ML model output. These catch patients who are trending towards disease before the ML model reaches its classification threshold:

Domain	Biomarker	Early Warning Threshold	Clinical Standard
Cardiovascular	Systolic BP	>= 120 mmHg	AHA Stage 1 Hypertension: 130 mmHg
Cardiovascular	Total Cholesterol	>= 180 mg/dL	Borderline High: 200 mg/dL
Metabolic	Fasting Glucose	100 – 125 mg/dL	ADA Pre-diabetes range
Metabolic	HbA1c	5.7 – 6.4%	ADA Pre-diabetes range
Renal	eGFR	60 – 89 mL/min	CKD Stage 2 (mildly decreased)
Renal	Creatinine	1.1 – 1.3 mg/dL	Above upper normal range for females

These thresholds are deliberately set BELOW the diagnostic criteria — they catch patients in the 'grey zone' before they become clinical cases, enabling preventive intervention.

13. Medical Knowledge Base

13.1 Structure

The KB is a Python dictionary with 20+ pre-written clinical entries covering the most common medical questions in the three disease domains. Each entry is a multi-line string containing evidence-based clinical information sourced from WHO, AHA, ADA, KDIGO, and NHS guidelines.

Category	KB Entries
Diabetes / Metabolic	metformin_details, diabetes_overview, hba1c_guide, insulin_resistance, sglt2_inhibitors, glp1_agonists
Cardiovascular	hypertension_basics, cholesterol_guide, cardiovascular_risk, statins_guide, ace_inhibitors
Renal / Kidney	nephrology_basics, creatinine_guide, ckd_management, dialysis_overview
Lifestyle	bmi_guide, smoking_cessation, exercise_recommendations, diet_diabetes
Medications	nsaid_warnings, metformin_details, furosemide_guide
Biomarkers	egfr_interpretation, hba1c_guide, creatinine_guide

13.2 Search Algorithm

KB search uses simple keyword matching with priority scoring. The search function tokenises the user's question and looks for keyword overlap with each KB entry key:

```
def search_kb(question: str) -> str:
    q_lower = question.lower()
    for key, content in MEDICAL_KB.items():
        keywords = key.replace('_', ' ').split()
        if any(kw in q_lower for kw in keywords):
            return content.strip()
```

This is intentionally simple. Complex semantic search (FAISS, BM25, embeddings) was evaluated but rejected because:

- The KB has < 30 entries — keyword matching is perfect for this scale
- No external dependencies needed
- Deterministic results — same question always gets same answer (no vector randomness)
- Sub-millisecond latency

14. OmniValidator — Clinical Range Validation

14.1 Purpose

Before any inference, all incoming vitals are validated against physiological limits. This prevents the ML models from receiving biologically impossible inputs that would produce meaningless risk probabilities.

If a doctor accidentally enters glucose = 6000 mg/dL (a typo for 600), the model should not silently predict maximum diabetes risk — it should alert the user to the data entry error.

14.2 Clinical Reference Ranges

25+ biomarkers are covered with min/max physiological limits. Selected examples:

Biomarker	Min	Max	Unit	Basis
Fasting Glucose	20	600	mg/dL	Below 20: hypoglycaemic coma. Above 600: hyperosmolar crisis threshold.
HbA1c	3.0	15.0	%	Below 3: haemolytic anaemia. Above 15: extremely rare even in DKA.
eGFR	3	150	mL/min	Below 3: effectively no kidney function. Above 150: measured maximum in hyperfiltration.
Systolic BP	60	250	mmHg	Below 60: cardiovascular collapse. Above 250: hypertensive emergency threshold.
BMI	10	80	kg/m2	Below 10: incompatible with survival. Above 80: extreme obesity outlier.
SpO2	70	100	%	Below 70: brain damage imminent. Above 100: physically impossible.

14.3 Normalisation

The normalise_vitals() function handles field name aliasing between the DB schema and ML model features:

```
# DB sends 'blood_pressure_sys' but model expects 'blood_pressure'
if 'blood_pressure_sys' in v and 'blood_pressure' not in v:
    v['blood_pressure'] = v['blood_pressure_sys']
```

This bridges the DB column naming convention and the legacy feature names from the original CSV datasets.

15. Complete Inference Pipeline — Step by Step

Here is the complete flow for a single POST /api/v1/diagnose/contextual request:

Step	Module	Operation	Output
1	Flask Router	Receive POST body, parse JSON	vitals dict, patient_id, patient_info
2	OmniValidator	Check all vitals against CLINICAL_RANGES	Pass / Fail (422 if fail)
3	patient_store	Load patient history from PostgreSQL (if patient_id given)	List of historical health scores
4	PatientContext	Build context: age_factor, sex_factor, lifestyle_factor, adaptive_weights, adaptive_thresholds	PatientContext object
5	_predict(heart)	Normalise → impute → scale → RF.predict_proba → calibrated prob	Heart risk [0.0–1.0]
6	_predict(diabetes)	Same pipeline with diabetes features	Diabetes risk [0.0–1.0]
7	_predict(kidney)	Same pipeline with kidney features	Kidney risk [0.0–1.0]
8	OmniExplainer	SHAP TreeExplainer on each model → merge → top-10 insights	Dict {feature: shap_value}
9	OmniScorer.compute_composite	Weighted sum → inversion → age ceiling → tier	health_score, risk_tier, domain_scores
10	OmniScorer.clinical_flags	Adaptive threshold checks + early warning biomarkers	List of flag objects
11	OmniAnalytics.calculate_trend	Delta from last score + ARIMA(1,1,0) forecast if >= 4 visits	trend_status, delta, forecast
12	OmniTriage.get_recommendation	Keyword + score-based urgency → max severity	department, urgency, urgency_note
13	OmniMedications.get_for_risks	Disease risks above threshold → medication suggestions	List of medication objects
14	OmniReadmission.compute_risk	5-component weighted scoring + context multipliers	readmission_risk_pct, level, timeframe
15	Flask Response	Assemble full result dict → jsonify → return 200	Complete clinical result JSON

16. Known Limitations

16.1 Dataset Limitations

- UCI Heart Disease dataset (Cleveland subset): 303 samples. This is extremely small for a production medical model. The model should be validated on larger datasets before clinical deployment.
- Pima Diabetes dataset: Women only, aged 21+, Pima Indian heritage. The model may not generalise well to other demographic groups without retraining.
- UCI CKD dataset: ~400 samples from a single hospital in India. Geographic and demographic generalisation is limited.

16.2 Static Models

The .pkl files are trained once and never updated from production patient data. As the platform accumulates real patient data, a retraining pipeline should be implemented to continuously improve model accuracy.

16.3 No Uncertainty Quantification

The model outputs a single probability value without confidence intervals. A production clinical system should output P(disease) ± confidence_interval to communicate prediction uncertainty to clinicians.

16.4 Feature Coverage

The diabetes model uses only 8 features (Pima dataset limitation). Real clinical diabetes prediction uses 15-20 features including HbA1c, fasting insulin, OGTT results, family history, and waist circumference. The current model is a reasonable approximation but not clinically complete.

16.5 LLM Memory

CHAT_SESSIONS is stored in-process memory. On Render free tier, the service restarts periodically, wiping all chat history. For production, chat sessions should be stored in Redis or PostgreSQL.

17. Conclusion

OmniV1 is a clinically-informed, mathematically-principled ML system that makes deliberate trade-offs appropriate for a medical MVP:

Design Decision	Choice Made	Alternative Rejected	Reason
Core Algorithm	Random Forest + Calibration	Neural Networks	Small datasets, interpretability requirements, deployment constraints
Calibration	Platt Scaling (sigmoid)	Isotonic Regression	More stable on small validation sets; mathematically smooth output
Imbalance Handling	SMOTE + class_weight=balanced	Random oversampling only	Dual protection; SMOTE generates diverse synthetic samples
Imputation	Median	Mean, KNN Imputation	Robust to outliers; computationally trivial; clinically appropriate
Decision Threshold	0.35 (adaptive)	0.50 (default)	Sensitivity > specificity for medical screening
LLM	Qwen 0.5B GGUF	GPT-4 API, Llama 7B	Memory constraints, privacy, cost, offline operation
Explainability	SHAP TreeExplainer	LIME, feature importance only	Exact Shapley values; directional; patient-specific
Forecasting	ARIMA(1,1,0)	Linear trend, LSTM	Stationary differencing captures trajectory; minimal data requirement

The system represents a production-ready clinical AI engine for an MVP deployment. Its architecture prioritises explainability, clinical safety, and resource efficiency over raw accuracy metrics — the correct priorities for a medical decision support system at this stage.

OmniV1 is not trying to replace clinicians. It is trying to give every clinician access to a second opinion — one that is available at 3am, never tired, and never forgets a patient's history.

— End of Document —